Appendix — Real-World AI Coding Agents: Cline, Roo Code & GitHub Copilot CLI

> 🎯 **Goal**: Understand how production AI agents implement the patterns you've learned throughout this syllabus.

> ⏱️ **Time**: 2–3h reading + exploration

> 📦 **Deliverable**: A mental model of how real-world coding agents are built and the key design decisions behind them

---

Table of Contents

1. Why Study Real-World Agents?
2. Cline — Deep Dive
3. Roo Code — Deep Dive
4. GitHub Copilot CLI — Deep Dive
5. Head-to-Head Comparison
6. Key Patterns to Learn From
7. Connection to Syllabus Concepts
8. Further Reading

Why Study Real-World Agents?

Throughout this syllabus, you learned to build agents from scratch — ReAct loops, planners, RAG pipelines, multi-agent systems. But how do **production agents** apply these same patterns at scale?

Cline and Roo Code are two of the most popular open-source AI coding agents. GitHub Copilot CLI is GitHub's terminal-native agent. They all read and edit your code, execute commands, and automate tasks — all powered by LLMs. Studying them reveals:

- How theoretical patterns (ReAct, planning, memory) translate to real products
- Engineering trade-offs in context management, tool design, and safety
- The emergence of **MCP (Model Context Protocol)** as a universal tool standard
- How mode/role-based architectures create specialized agent behaviors

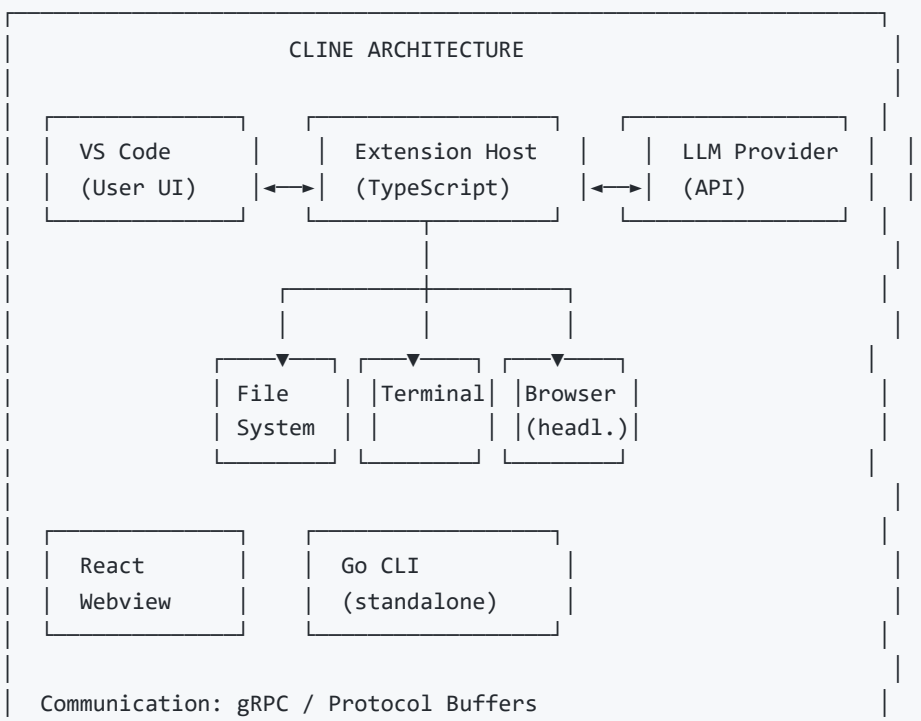
```
┌──────────────────────────────────────────────────────────────┐
│                    WHAT YOU'LL LEARN                         │
│                                                              │
│  Syllabus Theory          →    Real-World Implementation     │
│  ─────────────────             ──────────────────────        │
│  ReAct Loop (Day 3)       →    Cline's Plan/Act/Approve cycle│
│  Tools (Day 3)            →    MCP protocol & tool ecosystem │
│  Planner/Executor (Day 4) →    Roo Code's Architect + Code   │
│  RAG + Memory (Day 5)     →    Memory Bank & context windows │
│  Multi-Agent (Day 6)      →    Copilot CLI sub-agents & fleet│
│  Full System (Day 7)      →    Production-grade architecture │
│  Extensibility            →    Skills, plugins, custom agents│
│                                                              │
└──────────────────────────────────────────────────────────────┘
```

Cline — Deep Dive

What Is Cline?

Cline is an open-source autonomous coding agent that runs as a VS Code extension. Originally named "Claude Dev," it evolved into a model-agnostic agent with 5M+ installs. Its defining philosophy: **human-in-the-loop safety** — every action requires explicit user approval.

Architecture



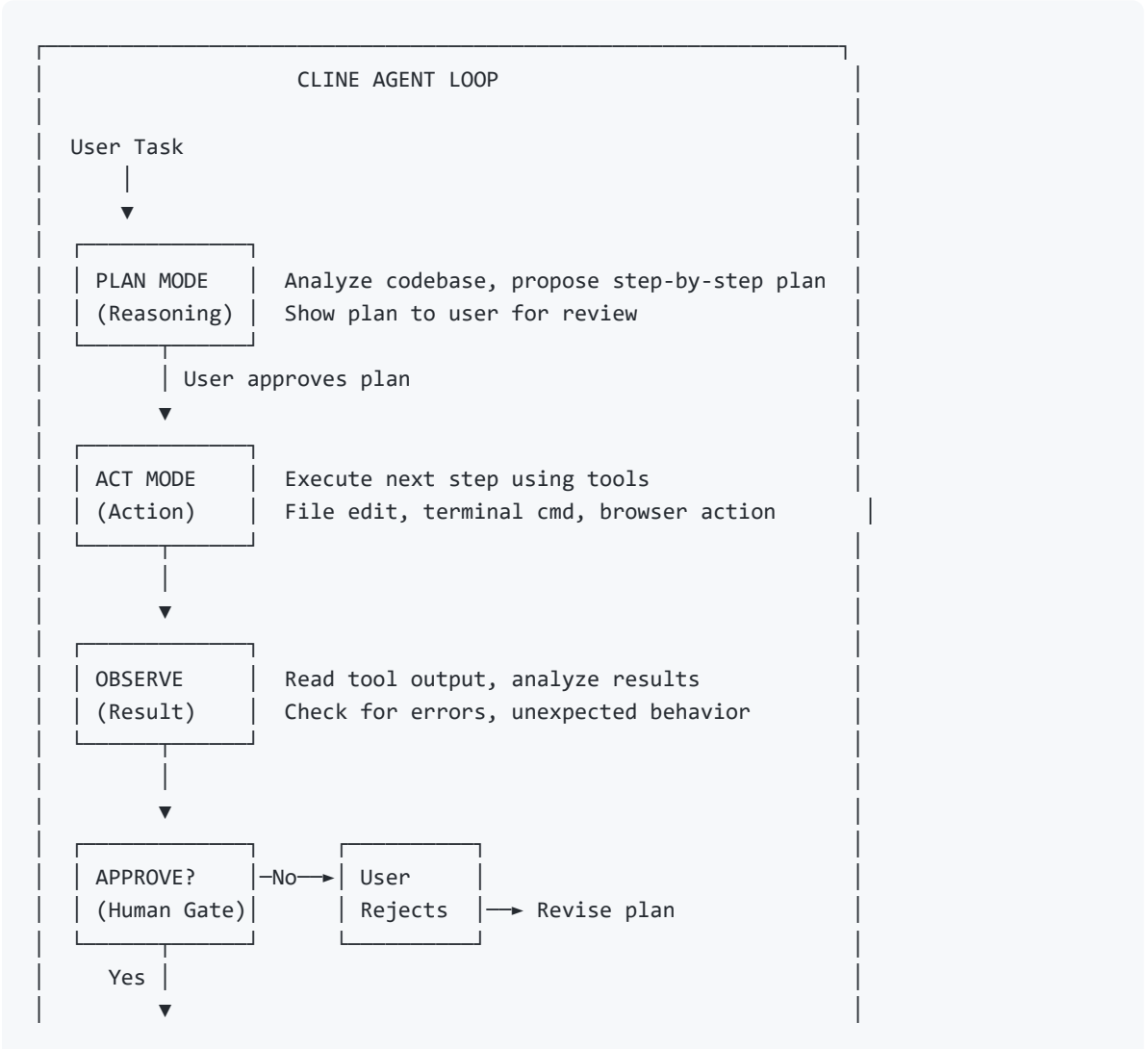


Key components:

Component	Technology	Purpose
Extension Host	TypeScript / Node.js	Core agent logic, tool orchestration
Webview UI	React / TypeScript	Chat interface, diff previews, approvals
CLI	Go	Standalone terminal usage without VS Code
Inter-process	gRPC + Protobuf	Communication between extension, CLI, and MCP servers

The Agent Loop

Cline implements a variation of the **ReAct pattern** (Day 3) with an explicit approval gate:



```
│    More steps? ──Yes──► Loop back to ACT MODE                │
│         │                                                    │
│         No                                                   │
│         ▼                                                    │
│     ✅ Done                                                  │
│                                                              │
└──────────────────────────────────────────────────────────────┘
```

Key insight: This is the ReAct loop from Day 3, but with a **human approval gate** inserted between every action. This makes Cline safer but slower — a fundamental design trade-off.

Tool System

Cline provides built-in tools that map directly to the tool concepts from Day 3:

```
# Conceptual representation of Cline's tool registry
CLINE_TOOLS = {
    # File operations
    "read_file":       {"input": "path", "output": "file_contents"},
    "write_to_file":   {"input": "path, content", "output": "success/fail"},
    "apply_diff":      {"input": "path, diff", "output": "patched_file"},
    "list_files":      {"input": "directory, recursive?", "output": "file_tree"},

    # Search
    "search_files":    {"input": "regex, path", "output": "matches"},  # ripgrep-bas
    "list_code_definitions": {"input": "path", "output": "AST_symbols"},  # tree-sit

    # Execution
    "execute_command": {"input": "command", "output": "stdout + stderr"},

    # Browser
    "browser_action":  {"input": "url, action, selector", "output": "screenshot + co

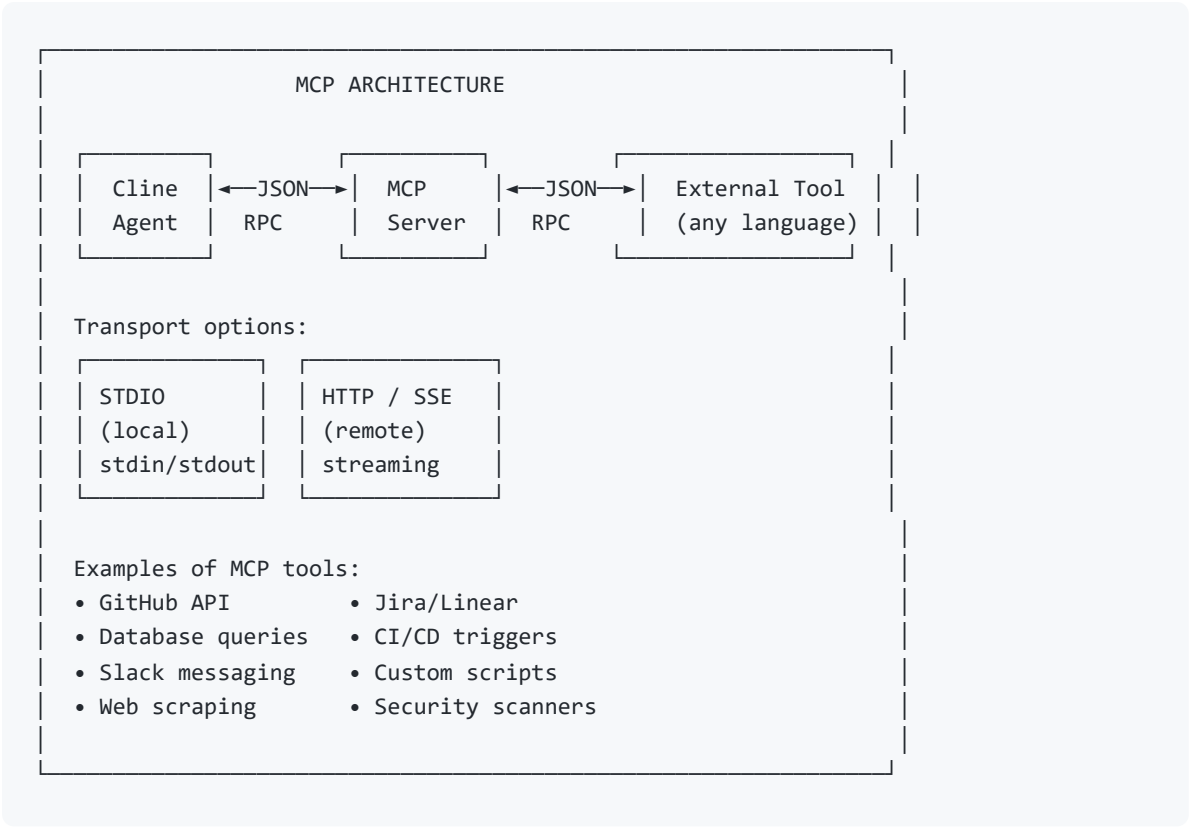
    # MCP (extensible)
    "use_mcp_tool":    {"input": "server, tool, args", "output": "tool_result"},
}
```

Search capabilities:

- **Regex search**: Uses `ripgrep` under the hood for fast codebase-wide pattern matching
- **AST-based search**: Uses `tree-sitter` to parse code and extract function/class/method definitions
- **File tree exploration**: Recursive directory listing with smart filtering

MCP — Model Context Protocol

MCP is one of the most important concepts to learn from these agents. It's a **universal standard for connecting LLMs to external tools** — think of it as "USB for AI tools."



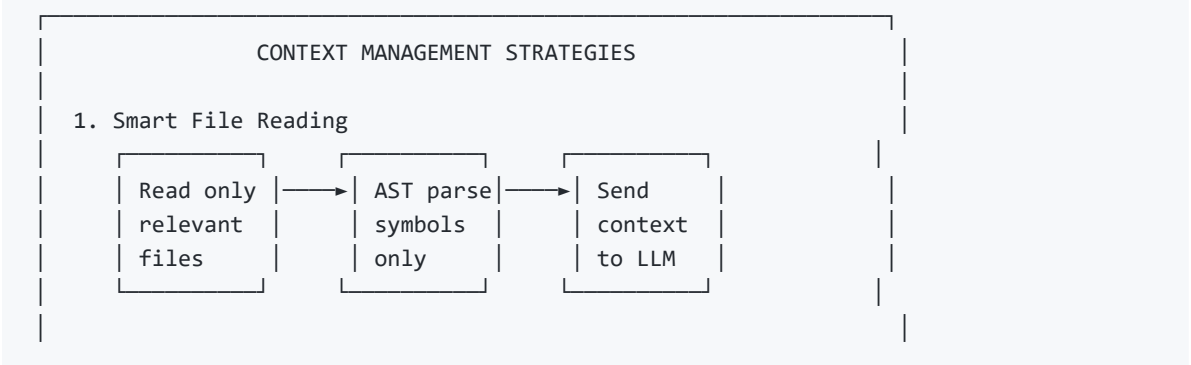
How MCP works:

1. An MCP server declares the tools it provides (name, description, input schema)
2. The agent discovers available tools at startup
3. When the LLM decides to use an MCP tool, it sends a JSON-RPC request
4. The server executes the tool and returns structured results
5. The agent incorporates results into its reasoning

Why this matters: MCP decouples tool implementation from the agent. Anyone can write an MCP server in any language, and any MCP-compatible agent can use it. This is the same concept as the tool abstraction from Day 3, but standardized across the industry.

Context Management

Cline uses several strategies to manage context efficiently:



```
│  2. Checkpoint System                                                  │
│     Every action creates a snapshot for rollback                       │
│     (like Git commits but for AI-driven changes)                       │
│                                                                        │
│  3. Context Window Slicing                                             │
│     Breaks large codebases into manageable chunks                      │
│     that fit within LLM token limits                                   │
│                                                                        │
└────────────────────────────────────────────────────────────────────────┘
```

Memory & Project Knowledge

Cline uses structured project files for persistent memory:

File	Purpose
`projectbrief.md`	High-level project goals and architecture
`progress.md`	What's been done, what's remaining
`techstack.md`	Technologies, frameworks, conventions
`currentTask.md`	Active task context and status

This is a practical implementation of the **memory types** from Day 3 (short-term, long-term, episodic) and the **RAG concepts** from Day 5.

Model Support

Cline is model-agnostic — it works with any LLM provider:

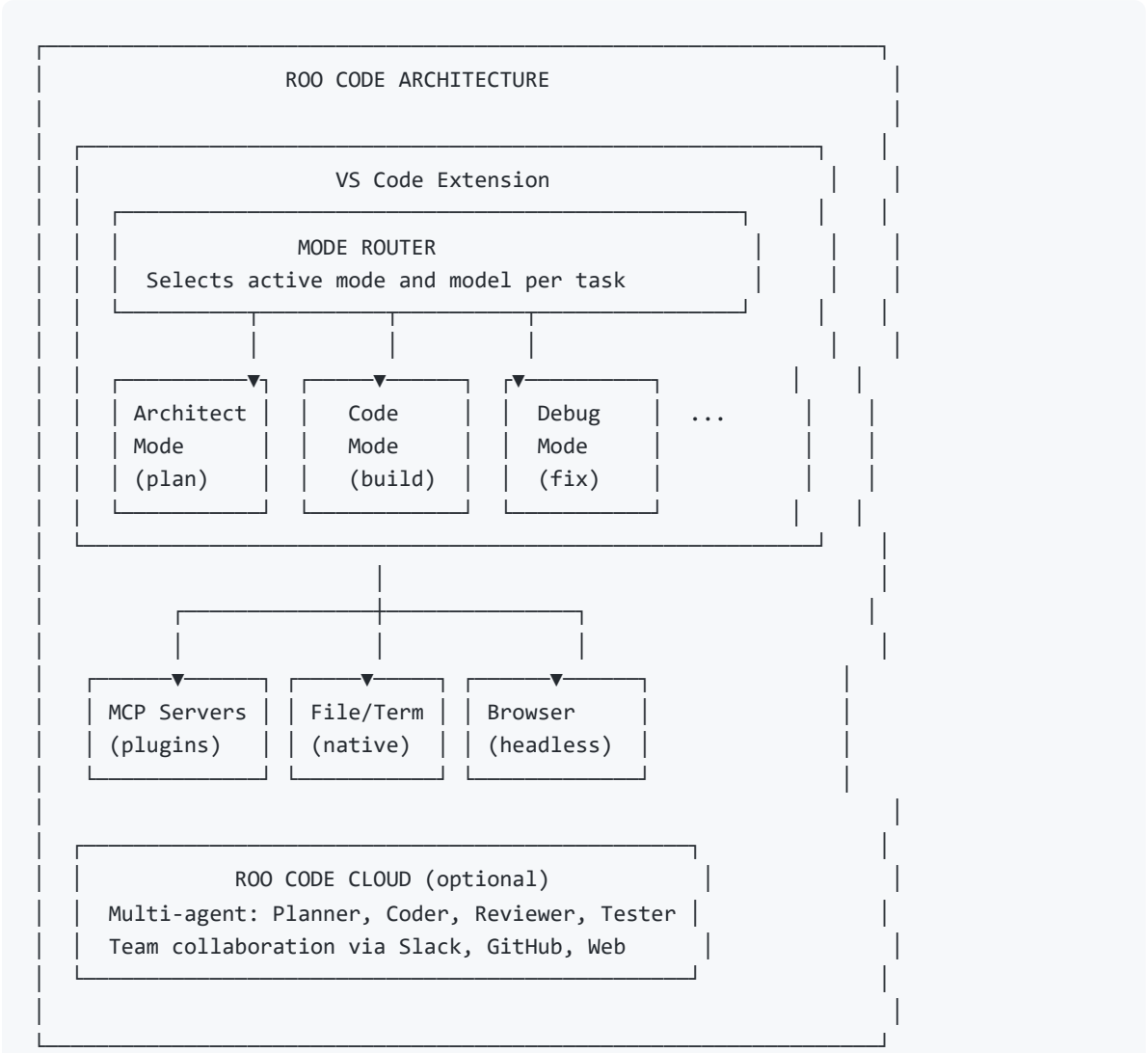
Provider	Models	Use Case
Anthropic	Claude Sonnet, Opus, Haiku	Primary coding models
OpenAI	GPT-4, GPT-4o	Alternative provider
Google	Gemini Pro, Flash	Cost-effective option
AWS	Bedrock (Claude, etc.)	Enterprise deployments
Local	Ollama, LM Studio	Privacy-first, offline use

Roo Code — Deep Dive

What Is Roo Code?

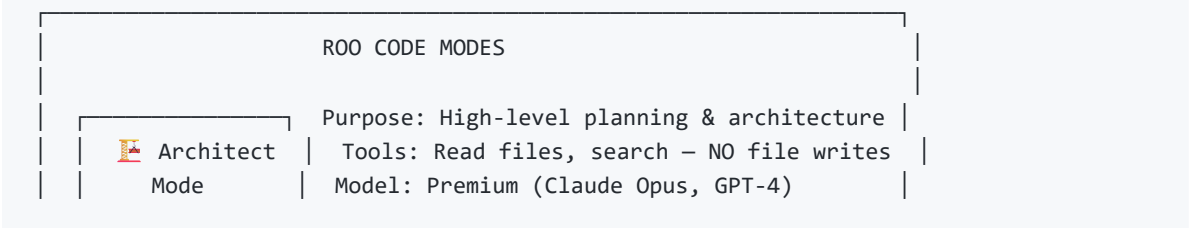
Roo Code (formerly "Roo Cline") is a fork of Cline that evolved into a **multi-agent, role-based coding platform**. While Cline focuses on safety-first single-agent interaction, Roo Code emphasizes **speed, specialization, and cost optimization** through its Modes system.

Architecture



The Modes System

Roo Code's most distinctive feature is its **Modes** — specialized agent personas, each with constrained behaviors and tool access. This is a direct implementation of the **agent roles** concept from Day 6 (Multi-Agent Collaboration).



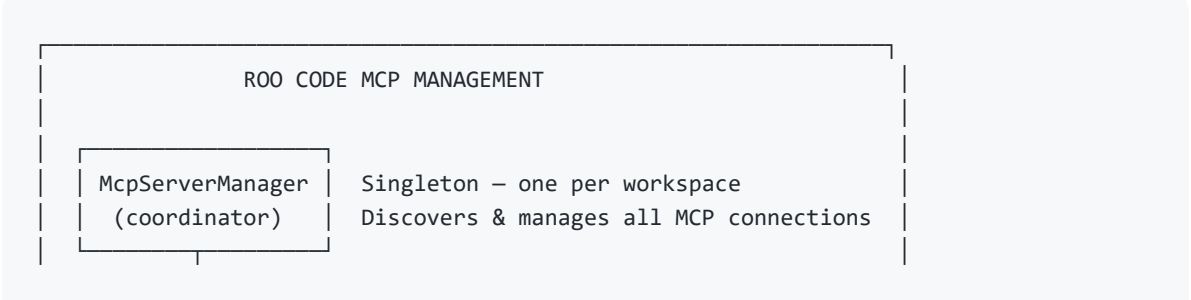


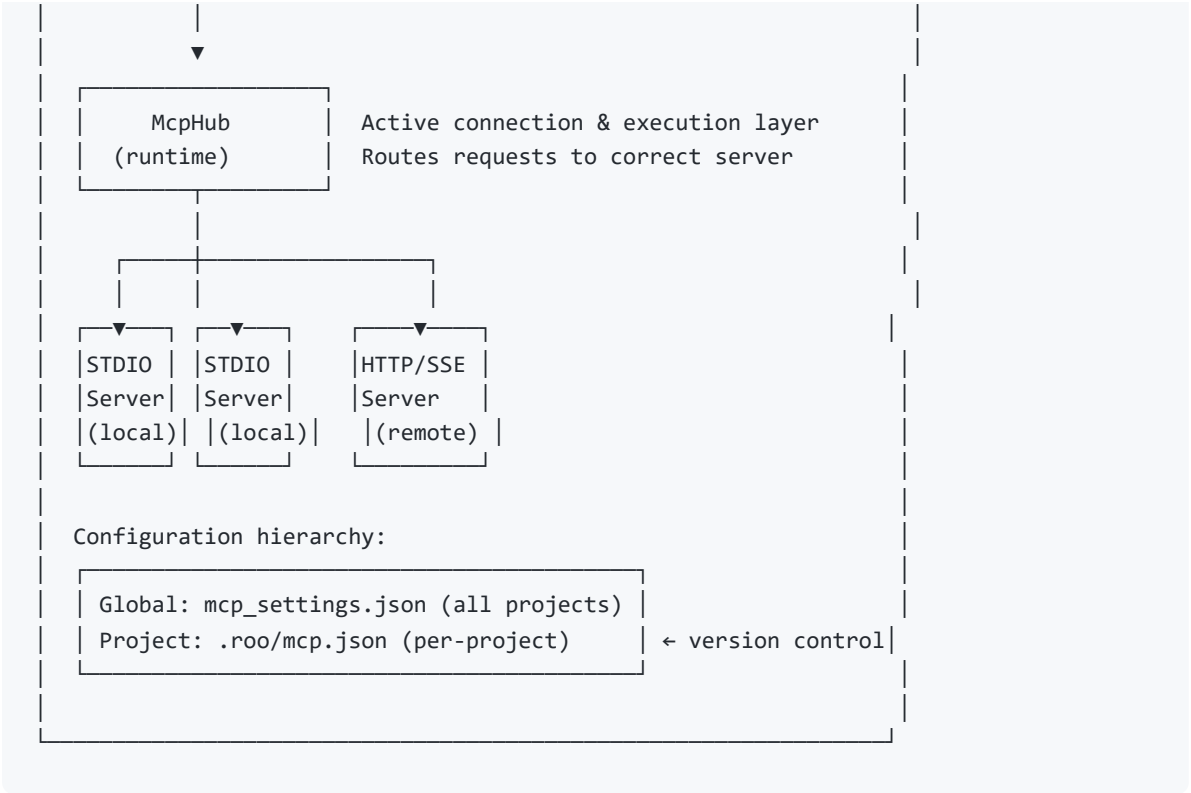
Why modes matter:

- Each mode has a **constrained tool set** — the Architect can't write files, preventing accidental changes during planning
- Each mode can use a **different LLM** — use expensive models for complex planning, cheap models for quick questions
- Custom modes let you create **domain-specific agents** (security reviewer, docs writer, performance optimizer)
- This is the Planner/Executor pattern from Day 4, but user-configurable and extensible

MCP Integration

Roo Code extends Cline's MCP support with a more structured management layer:



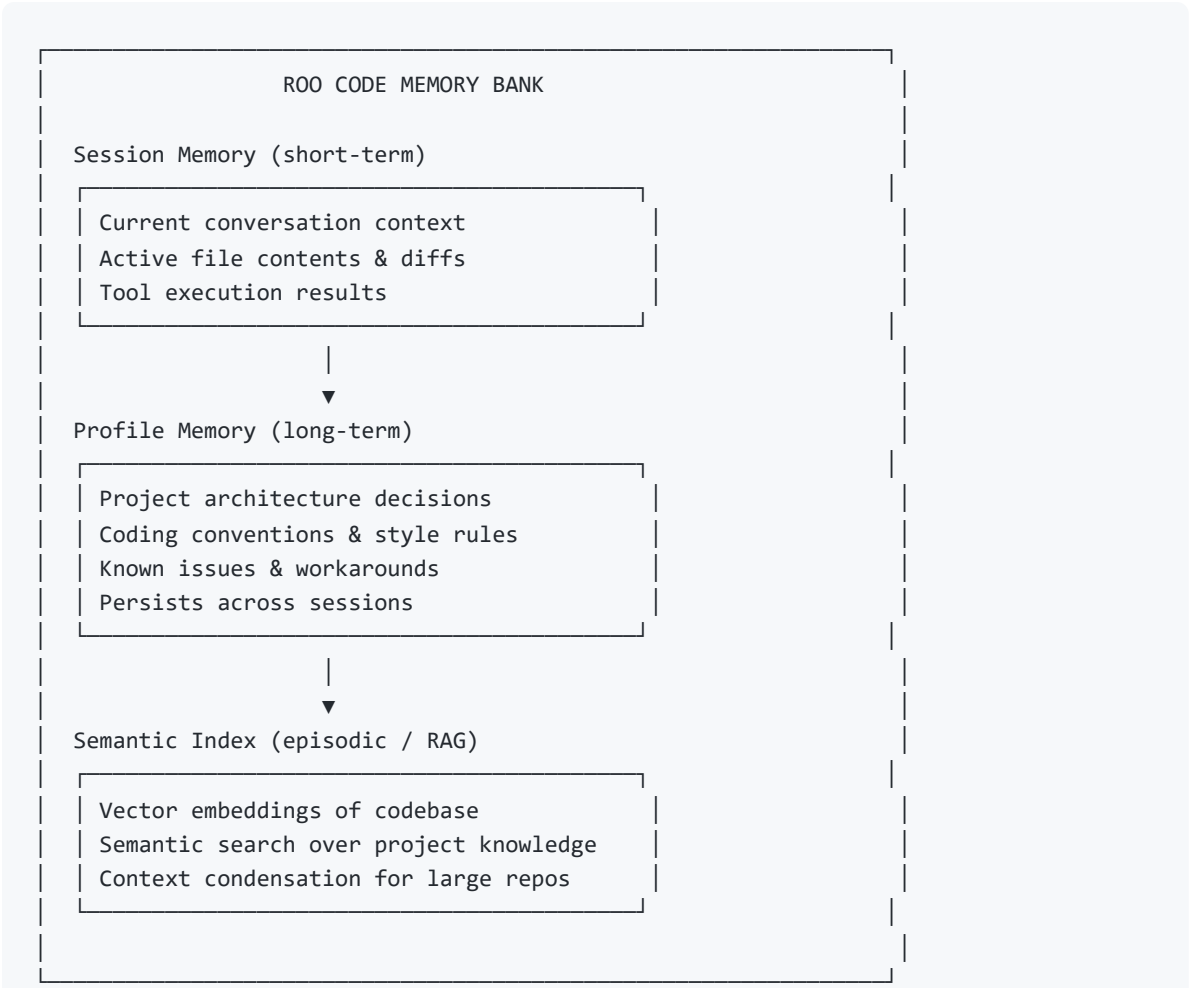


MCP configuration example:

```
{
  "mcpServers": {
    "github-tools": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-github"],
      "env": { "GITHUB_TOKEN": "${GITHUB_TOKEN}" }
    },
    "database-query": {
      "command": "python",
      "args": ["mcp_db_server.py"],
      "env": { "DB_URL": "postgresql://localhost/mydb" }
    },
    "remote-security-scanner": {
      "url": "https://security-scan.example.com/mcp",
      "transport": "http"
    }
  }
}
```

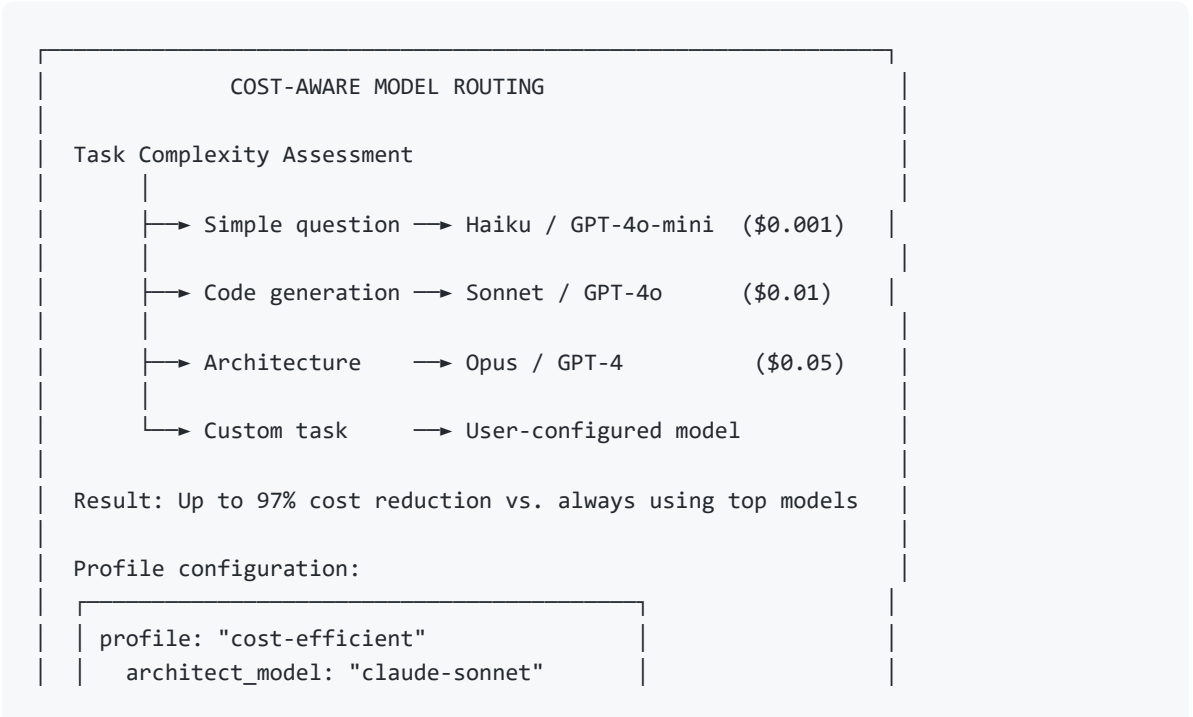
Memory Bank

Roo Code introduces **persistent memory** that survives across sessions — a production implementation of the memory concepts from Day 3 and the RAG pipeline from Day 5:



Cost Optimization

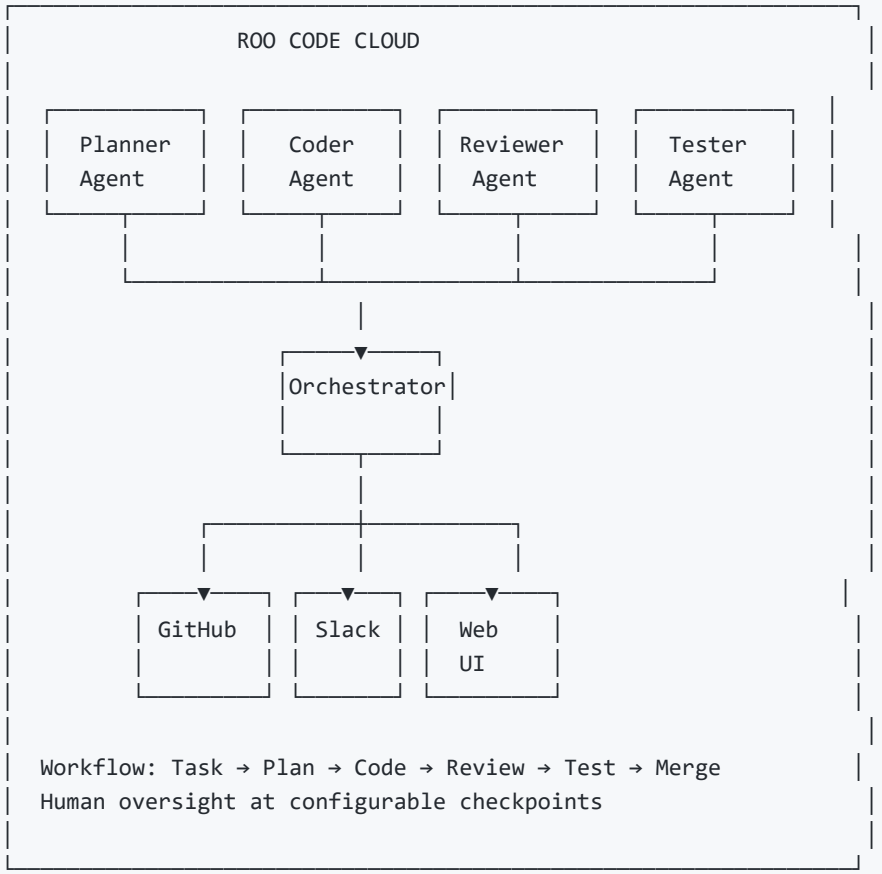
One of Roo Code's most practical innovations is **task-based model routing**:



```
│  │   code_model: "deepseek-coder"          │                  │
│  │   ask_model: "claude-haiku"             │                  │
│  │   debug_model: "claude-sonnet"          │                  │
│  └─────────────────────────────────────────┘                  │
│                                                               │
└──────────────────────────────────────────────────────────────┘
```

Roo Code Cloud

Roo Code Cloud extends the local extension into a **multi-agent collaboration platform** — the production version of Day 6's multi-agent system:



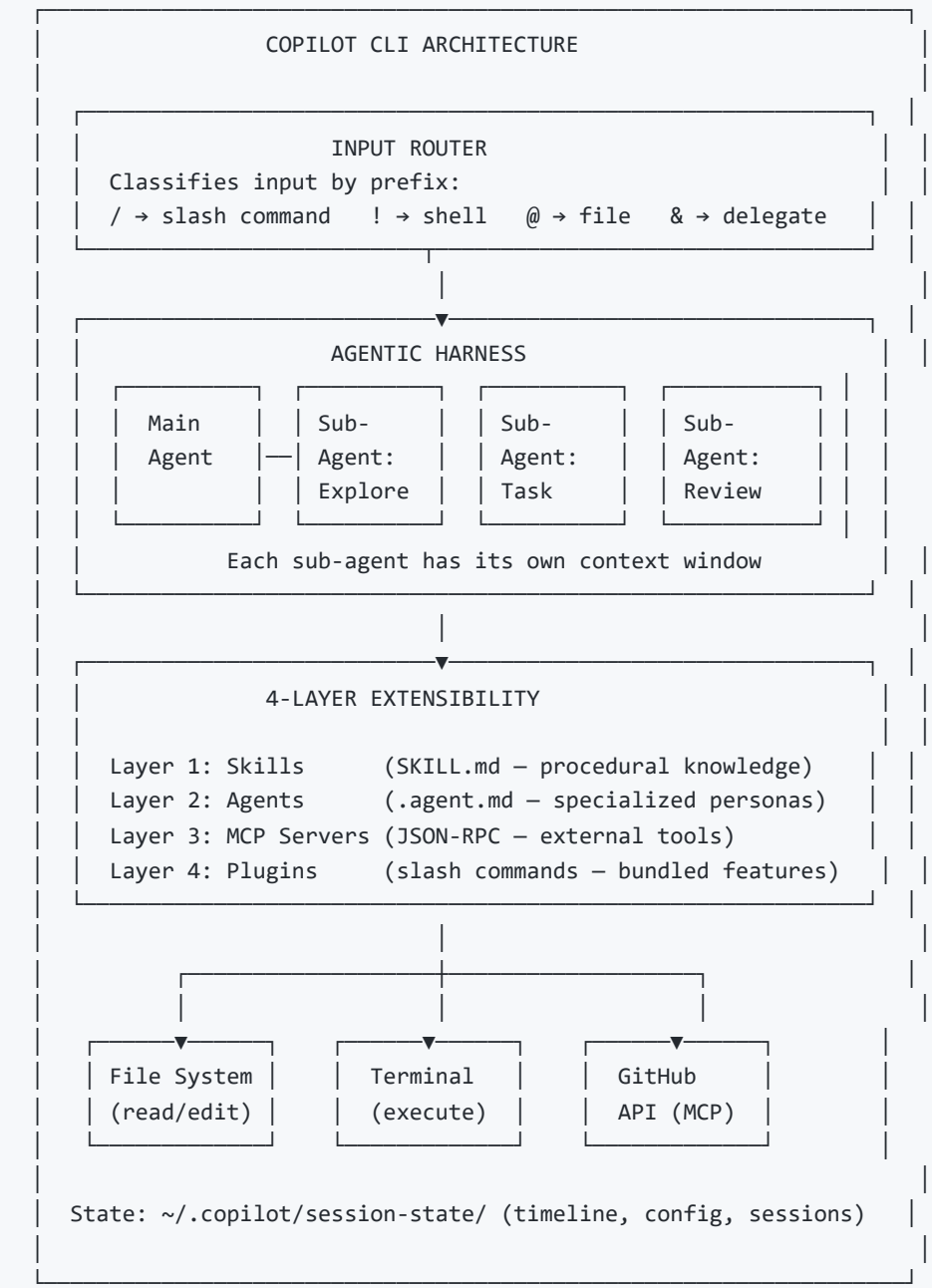
GitHub Copilot CLI — Deep Dive

What Is GitHub Copilot CLI?

GitHub Copilot CLI is a **terminal-native AI coding agent** — unlike Cline and Roo Code which live inside VS Code, Copilot CLI runs directly in your terminal. It's powered by the same agentic harness as GitHub's cloud-based Copilot coding agent, with deep GitHub ecosystem integration out of the box.

Key philosophy: **layered extensibility** — Skills, Custom Agents, MCP servers, and Plugins form a 4-layer system where each layer adds capabilities without requiring changes to the core agent.

Architecture



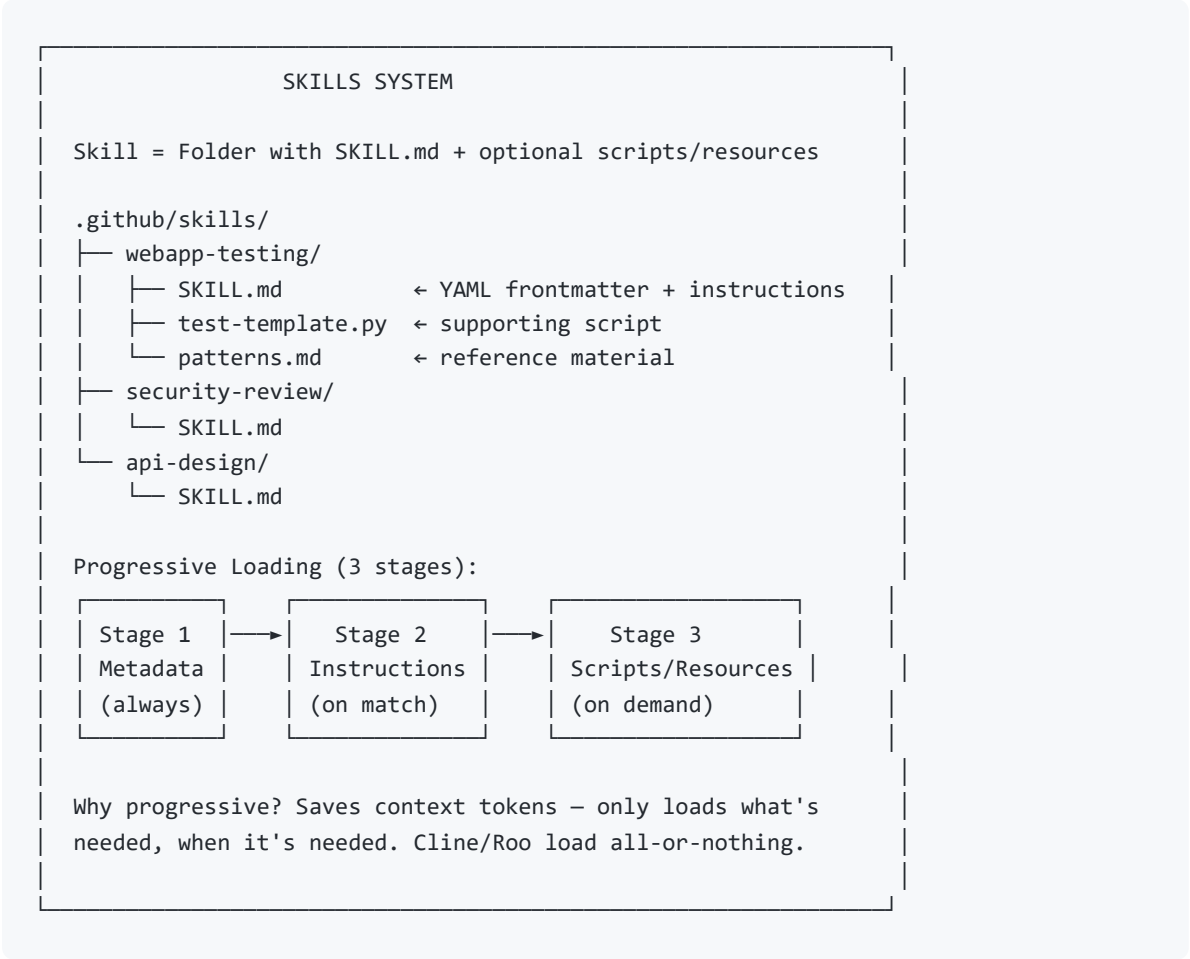
Key components:

Component	Purpose
Input Router	Classifies prompts by prefix (`/` , `!` , `@` , `&`) and routes to handlers

Agentic Harness	Core agent loop with model interaction, streaming, and tool execution
Sub-agents	Isolated context windows for explore, task, review, and general-purpose work
Session State	Persistent timeline of all actions, context, and decisions
GitHub MCP	Built-in MCP server for GitHub API (issues, PRs, repos, actions)

The Skills System

Skills are Copilot CLI's most distinctive feature — **portable, progressively-loaded packages of procedural knowledge**. Unlike MCP tools (which provide *actions*), Skills provide *instructions* — they teach the agent *how* to approach a domain.



SKILL.md example:

```
---
name: webapp-testing
description: Assists with web application test strategies and automated test creatio
---
```

```
## When to Use This Skill
Use this skill when the user asks about testing web applications,
writing test suites, or debugging test failures.

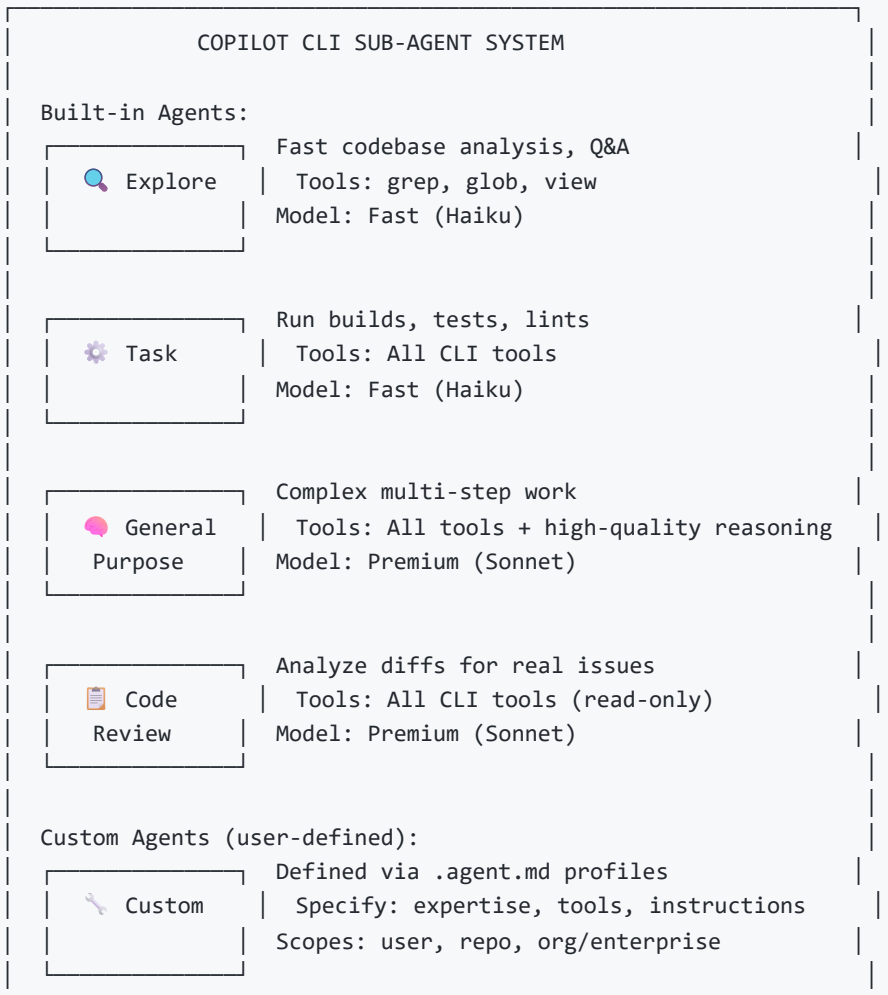
## Testing Strategy
1. Always start with unit tests for pure functions
2. Use integration tests for API endpoints
3. Use E2E tests sparingly — only for critical user flows

## Conventions
- Use pytest with fixtures for Python projects
- Use Jest + Testing Library for React projects
- Name test files with `.test.` or `.spec.` suffix
```

Key insight: Skills follow the open `agentskills.io` standard, making them **portable across agents** — the same skill works in Copilot CLI, Claude Code, and OpenAI Codex.

Custom Agents (Sub-agents)

Copilot CLI has a built-in **sub-agent architecture** — the main agent can delegate tasks to specialized sub-agents, each running in its own isolated context window:



```
│                                                                  │
│  Key difference from Roo Code modes:                             │
│  • Sub-agents run in SEPARATE context windows (no pollution)     │
│  • Main agent DELEGATES — it picks the right sub-agent           │
│  • Multiple sub-agents can run in PARALLEL (/fleet mode)         │
│                                                                  │
└──────────────────────────────────────────────────────────────────┘
```

Custom agent profile example (`.github/agents/security-reviewer.agent.md`):

```
---
name: security-reviewer
description: Reviews code changes for security vulnerabilities
tools:
  - grep
  - glob
  - view
  - powershell
---

You are a security expert. When reviewing code:
1. Check for injection vulnerabilities (SQL, XSS, command injection)
2. Verify authentication and authorization logic
3. Look for hardcoded secrets or credentials
4. Check for insecure cryptographic practices
5. Flag any unsafe deserialization
```

Fleet Mode (Parallel Sub-agents)

Copilot CLI's `/fleet` mode enables **parallel sub-agent execution** — multiple sub-agents work simultaneously on different parts of a task:

```
┌──────────────────────────────────────────────────────────────────┐
│                    FLEET MODE                                    │
│                                                                  │
│  Main Agent receives complex task                                │
│       │                                                          │
│       ├──► Sub-agent 1: "Explore auth module"     ──┐           │
│       │                                             │           │
│       ├──► Sub-agent 2: "Run test suite"          ──┤ Parallel  │
│       │                                             │           │
│       └──► Sub-agent 3: "Review recent changes"   ──┘           │
│                                                                  │
│       All results consolidated back to main agent                │
│       │                                                          │
│       ▼                                                          │
│  Main Agent synthesizes and continues                            │
│                                                                  │
│  vs. Roo Code: Modes are sequential (switch between them)        │
│  vs. Cline:    Single agent only (no delegation)                 │
```

```
│                                                                  │
└──────────────────────────────────────────────────────────────────┘
```

Delegation to Cloud Agent

A unique capability: Copilot CLI can **delegate tasks to GitHub's cloud-based coding agent**, creating a seamless local→cloud workflow:

```
┌──────────────────────────────────────────────────────────────────┐
│              LOCAL → CLOUD DELEGATION                            │
│                                                                  │
│  copilot CLI (local)                                             │
│       │                                                          │
│       │  /delegate "implement auth middleware"                   │
│       │  or: & implement auth middleware                         │
│       │                                                          │
│       ▼                                                          │
│  1. Commits unstaged changes as checkpoint                       │
│  2. Creates new branch                                           │
│  3. Hands off to Copilot coding agent (cloud)                    │
│       │                                                          │
│       ▼                                                          │
│  Cloud agent works autonomously:                                 │
│  • Opens draft PR                                                │
│  • Makes changes                                                 │
│  • Requests review when done                                     │
│       │                                                          │
│       ▼                                                          │
│  /resume — bring cloud session back to local CLI                 │
│                                                                  │
└──────────────────────────────────────────────────────────────────┘
```

Context Management

Copilot CLI uses several strategies that differ from the VS Code-based agents:

```
┌──────────────────────────────────────────────────────────────────┐
│            COPILOT CLI CONTEXT STRATEGIES                        │
│                                                                  │
│  1. Isolated Sub-agent Contexts                                  │
│     Each sub-agent has its OWN context window                    │
│     Main agent context stays clean                               │
│     (vs. Cline/Roo: single shared context)                       │
│                                                                  │
│  2. Progressive Skill Loading                                    │
│     Skills load in 3 stages: metadata → instructions → scripts   │
│     Only relevant skills consume context tokens                  │
│     (vs. Cline/Roo: no equivalent mechanism)                     │
│                                                                  │
│  3. Auto-compaction (/compact)                                   │
```

```
│     At 95% context limit, auto-summarizes history                │
│     Can also be triggered manually                                │
│                                                                   │
│  4. Custom Instructions Hierarchy                                 │
│     AGENTS.md, CLAUDE.md, GEMINI.md                               │
│     .github/copilot-instructions.md                               │
│     .github/instructions/**/*.instructions.md                     │
│     ~/.copilot/copilot-instructions.md                            │
│                                                                   │
│  5. Session Persistence                                           │
│     Full timeline stored in ~/.copilot/session-state/             │
│     Resume sessions with /resume or --continue                    │
│                                                                   │
└──────────────────────────────────────────────────────────────────┘
```

MCP Integration

Copilot CLI ships with the **GitHub MCP server built-in** — no configuration needed for GitHub API access. Additional MCP servers are added via `/mcp add`:

```
┌──────────────────────────────────────────────────────────────────┐
│            COPILOT CLI MCP SETUP                                 │
│                                                                  │
│  Built-in:                                                       │
│  ┌──────────────────────────────┐                                │
│  │  GitHub MCP Server           │  Issues, PRs, repos, actions   │
│  │  (pre-configured)            │  No setup required             │
│  └──────────────────────────────┘                                │
│                                                                  │
│  User-added (via /mcp add):                                      │
│  ┌──────────────────────────────┐                                │
│  │  Custom MCP Servers          │  Stored in                     │
│  │  (any JSON-RPC server)       │  ~/.copilot/mcp-config.json    │
│  └──────────────────────────────┘                                │
│                                                                  │
│  vs. Cline:    MCP supported, no built-in servers                │
│  vs. Roo Code: MCP + McpHub management layer                     │
│                                                                  │
└──────────────────────────────────────────────────────────────────┘
```

The 4-Layer Extensibility Model

This is the key architectural differentiator — Copilot CLI has **four distinct layers** for extending agent behavior, each solving a different problem:

```
┌──────────────────────────────────────────────────────────────────┐
│           4-LAYER EXTENSIBILITY MODEL                            │
│                                                                  │
│  Layer 4: Plugins                                                │
│  ┌──────────────────────────────────────────────────────────┐    │
```



Head-to-Head Comparison

Architecture & Stack

Aspect	Cline	Roo Code	Copilot CLI
Origin	Independent open-source	Fork of Cline, expanded	GitHub (proprietary)
Platform	VS Code extension	VS Code extension	Terminal-native
UI	React webview	React webview	Terminal (TTY)
CLI	Go-based standalone CLI	—	Primary interface

IPC	gRPC + Protobuf	VS Code API + MCP	MCP + session state
Cloud	—	Roo Code Cloud	GitHub Copilot coding agent

Agent Behavior

Aspect	Cline	Roo Code	Copilot CLI
Agent Pattern	Single ReAct agent	Multi-mode, role-based	Main agent + sub-agents
Planning	Plan mode (single agent)	Architect Mode (dedicated)	Plan mode + explore sub-agent
Execution	Act mode (same agent)	Code Mode (specialized)	Main agent or task sub-agent
Approval	Every action needs approval	Configurable (strict ↔ auto)	Per-tool approval, `/allow-all`
Custom Roles	Not supported	Fully customizable modes	Custom .agent.md profiles
Parallel Work	—	—	Fleet mode (parallel sub-agents)

Tools & Search

Aspect	Cline	Roo Code	Copilot CLI
File I/O	Read, write, diff	Read, write, diff	Read, write, diff
Terminal	Command execution	Command execution	Command execution
Browser	Headless automation	Headless automation	—
Code Search	Regex (ripgrep) + AST	Regex + AST + embeddings	Regex (ripgrep) + glob + LSP
MCP	Full support	Full support + McpHub	Built-in GitHub MCP + custom
Skills	—	—	Progressive SKILL.md system

Plugins	—	—	Slash command bundles

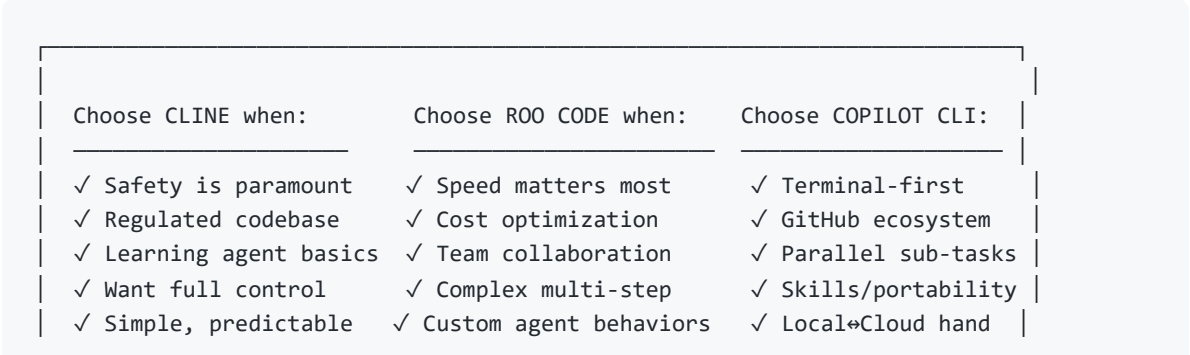
Context & Memory

Aspect	Cline	Roo Code	Copilot CLI
Context Strategy	AST parsing, smart windowing	Large windows, condensation	Sub-agent isolation + compaction
Memory	Per-session + project files	Memory Bank (cross-session)	Session state + custom instructions
Checkpoints	Manual (via approvals)	Session & profile snapshots	Session timeline + /resume
Semantic Search	—	Vector embeddings	LSP-based code intelligence
Progressive Loading	—	—	Skills load in 3 stages

Cost & Models

Aspect	Cline	Roo Code	Copilot CLI
Model Support	Multi-provider, configurable	Multi-provider, per-mode	Claude Sonnet, GPT-5 + others
Cost Control	Manual model selection	Task-based auto-routing	Premium request quota system
Local Models	Ollama, LM Studio	Ollama, LM Studio	— (cloud-only)
Cloud Delegation	—	Roo Code Cloud	/delegate to coding agent

When to Use Which?



```
│  ✓ Browser automation    ✓ Large codebase          ✓ Enterprise/org     │
│                                                                         │
└─────────────────────────────────────────────────────────────────────────┘
```

Key Patterns to Learn From

1. MCP as Universal Tool Protocol

All three agents use MCP, making it the emerging standard for AI tool integration. Key takeaway: **decouple your tools from your agent**. Write tools as MCP servers, and any agent can use them.

```
Agent-specific tools     →    MCP standard tools
─────────────────────          ──────────────────
• Locked to one agent          • Reusable across agents
• Custom integration           • Standard JSON-RPC
• Hard to share                • Easy to distribute
• One language                 • Any language
```

2. Mode/Role-Based Specialization

Roo Code's modes prove that **constrained agents outperform general-purpose ones**. Instead of one agent that does everything, create specialized agents with:

- Limited tool access (Architect can't write files)
- Optimized prompts (Debug mode has different system prompts than Code mode)
- Right-sized models (cheap models for simple tasks)

3. Human-in-the-Loop vs. Autonomous

Cline, Roo Code, and Copilot CLI represent different positions on the approval spectrum:

```
Cline              Copilot CLI                          Roo Code
│ ◄──────────────── APPROVAL SPECTRUM ──────────────────► │
│                        │                                  │
│  Every action    Per-tool approval     Most actions      │
│  approved        (granular control)    autonomous        │
│                                                          │
│  Safest          Balanced              Fastest           │
│  Slowest         Flexible              Riskier           │
```

Design lesson: Let users choose their position on this spectrum based on risk tolerance and task type.

4. Persistent Memory & Session Management

All three agents solve the "amnesia problem" differently:

- **Cline**: Structured markdown files (`projectbrief.md`, `progress.md`)
- **Roo Code**: Memory Bank with profiles and semantic indexing
- **Copilot CLI**: Session timeline persistence + custom instructions hierarchy

This connects directly to Day 5's RAG concepts — persistent memory is just RAG applied to the agent's own history.

5. Cost-Aware Model Routing

Roo Code's profile system demonstrates a critical production pattern: **not every task needs the best model**. Design your agent systems to route tasks to appropriately-sized models.

6. IDE-Native vs. Terminal-Native Agent Design

Cline and Roo Code are built into VS Code, while Copilot CLI runs in the terminal. Each approach has trade-offs:

```
VS Code Extension (Cline/Roo)          Terminal-Native (Copilot CLI)
──────────────────────────────         ─────────────────────────────
• Rich diff previews in editor         • No IDE dependency
• Browser automation (webview)         • Works over SSH / remote
• Inline code annotations              • Faster startup
• Native UI panels                     • Scriptable / composable
• Tight editor integration             • Works in any environment
```

Lesson: The best agent UX depends on the workflow — visual tasks favor IDE integration, automation and remote work favor terminal-native design.

7. Skills as Portable Knowledge Packages

Copilot CLI's Skills system introduces a pattern absent in Cline and Roo Code: **separating procedural knowledge from tool execution**. Skills teach the agent *how* to approach problems, while MCP tools give it *actions* to perform.

```
Skills (knowledge)              MCP Tools (actions)
──────────────────              ──────────────────
• "How to test APIs"            • "Run pytest command"
• "Security review rules"       • "Query GitHub API"
• "API design patterns"         • "Create Jira ticket"
• Progressively loaded          • All-or-nothing loaded
• Portable (agentskills)        • Portable (MCP standard)
```

Design lesson: The most powerful agents combine *knowledge* (what to do) with *tools* (how to do it). Consider separating these concerns in your own agent designs.

Connection to Syllabus Concepts

This table maps every concept from the 7-day syllabus to its real-world implementation in Cline, Roo Code, and GitHub Copilot CLI:

Syllabus Day	Concept	Cline	Roo Code	Copilc
Day 1: LLM Foundations	Tokenization, context windows	AST parsing, smart windowing	Context condensation, large windows	Sub-a isolati auto- compa
Day 1: LLM Foundations	Model selection	Multi-provider support	Per-mode model profiles	/mode comm per-ag model
Day 2: Prompt Engineering	System prompts, CoT	Mode-specific prompts	Role-based system prompts	Custor instruc hierar
Day 2: Prompt Engineering	Structured output	JSON tool call formatting	JSON tool calls + MCP schemas	JSON calls + schem
Day 3: ReAct Agent	Think → Act → Observe	Plan → Act → Approve cycle	Mode Router → Execute → Observe	Main a → dele → conso
Day 3: ReAct Agent	Tool definitions	Built-in tool registry	Built-in + MCP tools	4-laye + Age MCP + Plugin
Day 3: ReAct Agent	Memory types	Project files (short + long)	Memory Bank (all 3 types)	Sessio + instruc + skill
Day 4: Planner/Executor	Task decomposition	Plan mode proposes	Architect mode plans	Plan m explor

		steps		agent
Day 4: Planner/Executor	Critic loops	Human approval as critic	Review checkpoints	Code-sub-ag
Day 4: Planner/Executor	Guardrails	All actions need approval	Tool restrictions per mode	Per-to approv custor agent lists
Day 5: RAG + Memory	Vector search	—	Semantic index (embeddings)	LSP-ba code intellig
Day 5: RAG + Memory	Persistent knowledge	Markdown project files	Memory Bank + profiles	Skills (SKILL. custor instruc
Day 5: RAG + Memory	Chunking strategies	File-level context slicing	Context condensation	Progre skill lo (3 stag
Day 6: Multi-Agent	Agent roles	Single agent (all roles)	Architect, Code, Debug, Test	Explor Reviev Gener Custor
Day 6: Multi-Agent	Orchestration	—	Mode Router + Cloud	Fleet r (parall agents
Day 6: Multi-Agent	Communication	—	Cloud: Planner↔Coder↔Reviewer	/deleg cloud agent
Day 7: Full System	Production architecture	VS Code + gRPC + MCP	VS Code + MCP + Cloud	Termir MCP + GitHul agent
Day 7: Full System	Safety & governance	Human-in-the-loop	Configurable approval + SOC2	Per-to approv org-le agents

Day 7: Full System	Scalability	Single user, local	Team collaboration, cloud	/deleg /resun (local

---

Further Reading

Cline

- **GitHub**: github.com/cline/cline
- **VS Code Marketplace**: Cline Extension
- **DeepWiki Architecture**: deepwiki.com/cline/cline
- **Community**: Discord (linked from GitHub)

Roo Code

- **GitHub**: github.com/RooCodeInc/Roo-Code
- **Documentation**: docs.roocode.com
- **MCP Guide**: docs.roocode.com/features/mcp
- **Roo Code Cloud**: roocode.com

GitHub Copilot CLI

- **GitHub**: github.com/github/copilot-cli
- **Documentation**: docs.github.com/copilot
- **Custom Agents Guide**: docs.github.com — Custom Agents
- **Skills Guide**: docs.github.com — Skills
- **DeepWiki Architecture**: deepwiki.com/github/copilot-cli
- **Agent Skills Standard**: agentskills.io

MCP Standard

- **Specification**: modelcontextprotocol.io
- **Server Examples**: github.com/modelcontextprotocol

Comparisons

- Cline vs Roo Code vs Cursor (Better Stack)
- Roo Code vs Cline (Qodo)
- Roo Code vs Cline (Openxcell)

← Back to Syllabus Overview